Capability Loss Under Reward-Driven Fine-Tuning

A Preliminary Study on Procedural Mazes

Tejas Naladala
University of Washington (Undergraduate)
tejas.naladala@gmail.com · github.com/tejasnaladala/maze-rl-baselines
Working draft v0.1, April 2026

Abstract. On 9x9 procedural mazes generated by depth-first search, we observe three empirical results worth documenting. First: across seven HP-tuned modern reward-driven configurations (Stable-Baselines3 PPO, DQN, A2C across three learning rates each, 70 runs total), the best baseline (SB3 DQN at default LR) reaches 31.4% test success, statistically tied with a uniform random walk (32.7%); PPO, A2C, and recurrent DQN underperform the random walk by significant margins. A behavior-cloning MLP of identical architecture, observation, and optimizer reaches 97.4%. Second: initializing DQN online and target networks from the BC-distilled weights (which evaluate at 97.2% before fine-tuning) and fine-tuning with the same shaped reward as the from-scratch DQN collapses test success to 13.6% across all five random seeds, ending below the from-scratch DQN baseline at any tested learning rate. Third (added in response to an adversarial audit recommendation): adding a standard count-based intrinsic-motivation bonus to PPO across 20 seeds at 500K env steps each yields mean 9.4% test success (sd 18.2, median 3.0, range 0 to 68), with 10 of 20 seeds collapsing to 0 to 2% and only 2 of 20 reaching above 50%. The exploration-augmented baseline does not close the gap to BC distillation in mean. We frame the BC -> DQN warm-start collapse as a candidate instance of capability loss under reward-driven fine-tuning, with the caveat that this is one fine-tune recipe at n=5; broader robustness is open. The result is preliminary and has known scope limitations, most prominently that DFS-generated mazes are tree graphs on which a wall-following heuristic is provably complete (it terminates at the goal on every instance, though not at shortest-path-optimal step count). The contribution of this note is to document the result, state the scope limits clearly, and identify the experiments that would resolve them. Code, data, manifest, and seeds: github.com/tejasnaladala/maze-rl-baselines.

1 Findings

1.1 Setup

9x9 procedural mazes generated by depth-first-search with random tree backtracking. Each maze has a single start cell and a single goal cell. Observation is a 24-dimensional egocentric feature vector (local 3x3 wall and hazard flags, normalized Manhattan distance and direction to goal, last-3 action history, visit counts on adjacent cells). Action space is {N, S, E, W} on a deterministic transition function. Shaped reward (+10 on goal, small per-step penalty, hazard penalty, revisit penalty). Episode terminates at 4*n^2 steps where n is the maze side (324 steps at 9x9).

We compare five algorithm families: a uniform random walk and a no-backtracking random walk (1-bit memory); an egocentric left-hand-rule heuristic (5 lines of Python); a behavior-cloning MLP trained on (state, action) pairs from the BFS oracle; six reward-driven RL configurations on the same audited harness (custom MLP_DQN and DRQN at default LR, plus SB3 PPO/DQN/A2C across multiple LRs); and a BC -> DQN warm-start where DQN online and target networks are initialized from BC weights and fine-tuned with reduced exploration. Total compute: approximately 50 GPU-hours on

consumer hardware (RTX 5070 Ti) plus rented H200 cluster time on vast.ai, across roughly 4,200 archived result records. Full hyperparameter sweep specification in Table 2 (Section 4).

1.2 Headline Numbers

Method	Test Success Rate (95% CI)
Wall-following heuristic (oracle, see §1.4)	100.0%
BC_MLP (oracle distillation, n=20)	97.4% (sd 2.5)
NoBackRandom (1-bit memory random walk, n=50)	51.5% (sd 6.6)
Random walk (uniform, n=50)	32.7% (sd 6.1)
SB3 DQN, default LR (best HP, n=10)	31.4% (sd 7.2)
MLP_DQN, custom (n=40)	19.3% (sd 6.7)
DRQN, custom LSTM (n=40)	19.0% (sd 10.8)
A2C, SB3 (n=10)	8.4% (sd 4.3)
PPO, SB3 (best HP, n=10)	6.0% (sd 6.9)
Count-based PPO (intrinsic, n=20)	9.4% (sd 18.2, median 3, range 0 to 68)
BC -> DQN (post fine-tune, n=5)	13.6% (per-seed: 0, 12, 16, 18, 22)

Table 1. Test success rates on held-out 9x9 procedural mazes (50 mazes per training seed, drawn from a disjoint seed range). Standard deviations across seeds. Best HP per algorithm is selected from a 3-LR sweep; modern (SB3) baselines and custom baselines reported separately. The best of seven HP-tuned modern reward-driven configurations (SB3 DQN, default LR) is statistically tied with uniform Random; PPO, A2C, and the custom MLP_DQN/DRQN underperform Random by significant margins.

The first row is a topological observation, not an empirical claim about heuristic intelligence; see §1.4. The remaining rows are the central results of this note. The best HP-tuned modern reward-driven baseline (SB3 DQN at default LR, 31.4 percent) is statistically tied with uniform Random (32.7 percent). The BC-distilled MLP, using the same 24-d observation and same 24-64-32-4 architecture as MLP_DQN, exceeds it by 64.7 percentage points.

1.3 BC -> DQN Warm-Start Collapse

DQN initialized from BC_MLP weights begins evaluation at 97.2% test success, matching BC_MLP within bootstrap CI. After training under the standard DQN reward signal, all five seeds collapse to test success in the [12.4%, 14.9%] range, mean 13.6%. The collapse is monotonic in environment steps in four of five seeds; the remaining seed shows non-monotonic descent with the same end-point. This is the main empirical finding of the note.

Figure 1. Test success rate vs. environment steps for the BC -> DQN warm-start, five seeds. Available at github.com/tejasnaladala/maze-rl-baselines.

1.4 Calibration: Where the BC Oracle is Trivial

The 5-line wall-following oracle solves 100% of these mazes. This is not an empirical claim; it is a topological fact. DFS-generated mazes are tree graphs, and on tree graphs the right-hand rule is provably complete (it terminates at the goal on every instance) but not provably optimal in step

count. In our loopy-maze pilot (Wilson's algorithm with extra passages, hazards disabled), the wall-follower still solves 100 percent at a mean 21.7 steps vs the BFS oracle's 12.2 steps; the heuristic is correct but not shortest-path. The oracle's 100 percent success on tree mazes therefore tells us we are testing in a regime where a topologically simple heuristic suffices, which is also the regime where a BC distillate of that heuristic looks unusually strong.

This is the most important scope limitation of the present results. Whether the observed capability hierarchy (BC much greater than every reward-driven RL baseline, with BC -> DQN warm-start collapsing back below baseline) survives on cyclic maze families is an open question. We discuss the proposed experiment in §2.1.

2 Open Questions and What Would Resolve Them

We list the unresolved questions in approximate order of importance, with the experiments that would resolve each and rough compute estimates. The bait of this note is that all five questions are tractable; none requires resources beyond a few hundred H200-hours and the right collaborator.

2.1 Q1. Tree-Maze Artifact

DFS produces tree mazes. The right-hand rule is no longer perfect on cyclic graphs. The empirical question is whether the observed hierarchy and the BC->DQN collapse survive when the maze graph contains cycles, multiple paths between cells, or disconnected wall regions.

- **Proposed experiment.** Wilson's algorithm with random extra edges (parametrized by added-edge fraction); Kruskal generation with cycle injection; hand-crafted MultiRoom layouts with explicit loops. For each generator family, repeat the full 5-algorithm comparison and the BC -> DQN warm-start protocol. Report the BC oracle's success rate as a calibration row alongside the algorithm comparisons.
- **Compute.** Approximately 50 H200-hours for a full sweep across three generator families.
- **Hypothesis.** The BC much greater than Random gap shrinks but does not vanish (BC still distills a non-trivial policy). The BC -> DQN collapse persists, because the bootstrap-instability mechanism we conjecture below is not specific to tree topology. Not yet tested.

2.2 Q2. Mechanism Isolation

Three candidate mechanisms for the BC -> DQN collapse are not yet distinguished by data:

1. **Bootstrap instability.** TD-target updates with function approximation under sparse reward have known divergence properties (Baird, 1995; Tsitsiklis and Van Roy, 1997). The BC-induced minima may not be fixed points of the bootstrap operator.
2. **Distribution shift.** As DQN's epsilon-greedy explores, the replay buffer drifts from the BC trajectory distribution toward random trajectories, and the value function regresses toward random-walk-like estimates.
3. **Reward shape.** The per-step penalty interacts adversely with the BC-induced policy's local exploration patterns.

- **Proposed experiments.** (a) Replace the TD target with the oracle's action distribution under the same training loop (i.e., distillation without bootstrap). If the collapse vanishes, bootstrap is

isolated. (b) Freeze the replay buffer at the BC-induced distribution and continue DQN updates. If the collapse persists, distribution shift is not the primary cause. (c) Set the per-step penalty to zero. If the collapse magnitude shrinks meaningfully, reward shape is a contributor.

- **Compute.** Approximately 30 H200-hours for the three ablations across five seeds each.
- **What a collaborator could add.** Loss-landscape characterization tools: Hessian eigenspectrum near the BC minimum under the bootstrap loss; basin-volume estimates; NTK or singular-learning-theory framing. We have weights, trajectories, and seeds. We do not have the theoretical machinery to extract a clean story from them.

2.3 Q3. Algorithmic Scope

If the diagnosis is bootstrap instability, then offline RL algorithms designed to control bootstrap error (CQL, IQL, Decision Transformer) should preserve BC-initialized performance.

- **Proposed experiment.** Re-run the BC -> fine-tune protocol with CQL, IQL, and DT replacing DQN, holding everything else fixed.
- **Compute.** Approximately 80 H200-hours.
- **What a collaborator could add.** This is the wheelhouse of researchers working on offline RL with conservative value estimation. The result either further isolates the bootstrap mechanism or surfaces a deeper issue.

2.4 Q4. Continuous Control and Language Analogues

The structural pattern (pretrained capability + reward-driven fine-tuning + capability loss) is the analogy sometimes drawn to RLHF on language models. The present data does not provide direct evidence for the analogy. The pattern is suggestive, not demonstrated.

- **Proposed experiment.** A small-scale RLHF setup where pretrained capability is independently verifiable (math-style tasks where correctness is checkable), the reward signal is deliberately mis-specified (rewarding length or format rather than correctness), and the same paired comparison (initial capability vs. post-fine-tune capability) is measured.
- **Compute.** Significantly larger than Q1-Q3, depending on model size. This is the experiment most plausibly out of scope for a solo undergraduate effort and is exactly where a frontier-model lab collaborator would be the multiplier.

2.5 Q5. Theoretical Framing

The empirical observation is consistent with several theoretical frames: bootstrap instability with function approximation, loss-landscape sharpness leading to non-preserved minima under different gradient sources, singular learning theory's account of phase transitions during training. None of these is directly tested by the current data.

- **What a collaborator would add.** Whichever frame they bring. The empirical setup is small enough to support targeted analyses (Hessian computations, NTK regression, fixed-point characterization of the bootstrap operator). The marginal value of theoretical insight on this small system is high.

3 Why This Matters (Scoped)

One concrete methodological implication. Same-architecture, same-observation, same-optimizer behavior-cloning controls should be standard alongside reward-driven RL baselines. The 97% to 14% capability loss documented here is invisible to a researcher who does not run the BC baseline; the training reward continues to increase while test success collapses. This is a cheap diagnostic to add and we recommend it as standard practice for any RL benchmark with sparse or shaped reward.

One careful claim. The result is a candidate minimal mechanistic analogue of capability loss under reward-driven fine-tuning of a previously-competent policy. Whether the same dynamic operates in larger systems (RLHF on language models, sim-to-real with reward shaping) is an open empirical question that the present data does not settle. The structural similarity is suggestive enough to be worth testing carefully (Q4 in §2). It is not strong enough to be cited as direct evidence about LLMs, and we do not so cite it.

4 Methods

4.1 Environment

9x9 procedural mazes generated by depth-first-search with random tree backtracking. Train and test mazes are drawn from the same distribution but disjoint random-seed ranges; test seeds are deterministic per training-seed via the harness function main_sweep_test_seeds(seed, 50). Observation is a 24-dimensional egocentric feature vector (local 3x3 wall and hazard flags, normalized Manhattan distance and direction to goal, last-3 action history, visit counts on adjacent cells). Action space {N, S, E, W} on a deterministic transition function. Shaped reward: +10 on goal, small per-step penalty, hazard penalty, revisit penalty. Episode termination at 4*n^2 steps. Test set: 50 held-out mazes per training seed.

4.2 Algorithms and Hyperparameter Sweep

Table 2 summarizes the hyperparameter sweep. The SB3 modern-baseline sweep (PPO, DQN, A2C across 3 learning rates each at 10 seeds per cell) is the 70-run direct response to the standard reviewer concern that DQN-family baselines may be under-tuned. The custom MLP_DQN and DRQN baselines used in the broader headline tables ran at 40 seeds each at default LR. BC ran 20 seeds; BC -> DQN ran 5 seeds at SB3 DQN default LR. Each reward-driven run was 500,000 environment steps (SB3) or 200,000 environment steps (custom and BC -> DQN). BC ran on a fixed (state, action) dataset of approximately 8,800 pairs per seed collected from the BFS oracle on the training distribution.

Algorithm	Learning Rates	Exploration	Network	Seeds × LRs
SB3 DQN	1e-4, 5e-4, 1e-3	epsilon 1.0 -> 0.05	MLP 24-64-32-4	10 × 3
SB3 PPO	1e-4, 3e-4, 1e-3	entropy = 0.01	MLP 24-64-32-4	10 × 3
SB3 A2C	7e-4 (default)	entropy = 0.01	MLP 24-64-32-4	10 × 1
MLP_DQN (custom)	5e-4	epsilon 1.0 -> 0.05	MLP 24-64-32-4	40 seeds
DRQN (custom)	5e-4	epsilon 1.0 -> 0.05	LSTM(64) + MLP	40 seeds
BC	1e-3	n/a (cross-entropy)	MLP 24-64-32-4	20 seeds
BC->DQN	5e-4	epsilon 0.20 -> 0.05	MLP 24-64-32-4	5 seeds

Table 2. Hyperparameter sweep. Total: 70 SB3 modern-baseline runs (PPO/DQN/A2C × 3 LRs × 10 seeds) + 80 custom DQN/DRQN runs + 20 BC seeds + 5 BC -> DQN seeds + the deterministic baselines (oracle, heuristic, random, NoBackRandom). Approximately 4,200 archived result records across all experiments, every record SHA-256 manifested and code-hash pinned.

4.3 Statistical Analysis

Per-policy success rate is mean across seeds at the best learning rate (modern baselines) or at default LR (custom baselines). Standard deviations are across seeds. Comparisons between policies use a paired bootstrap stratified by seed (10,000 resamples) with Holm-Bonferroni family-wise error correction; we also report Cohen d effect sizes and Bayesian posterior probabilities under Beta-Binomial conjugate priors. The headline ordering is posterior-certain at the 0.001 level for every reported pairwise comparison. Full statistical pipeline in stats_pipeline.py.

5 Status, Constraints, and Collaboration

This is a v0.1 working draft from a solo undergraduate research effort at the University of Washington (B.S. Electrical & Computer Engineering and Applied Mathematics, expected 2028). It reflects approximately three months of full-time work, paid out of pocket on rented spot H200 compute via vast.ai and Jarvis Labs.

What we have. A reproducible empirical setup; full hyperparameter configurations, training trajectories, and seeds public on GitHub; a working harness for the proposed Q1-Q3 experiments; pre-registered protocols on related work in the author's broader research line, including a public deviation log on a related Sobol variance-decomposition pilot that violated its own pre-registered consistency check (Parchment Labs, separate paper). Three months of operational experience running this kind of empirical study at single-author scale.

Constraints we are honest about. (1) *Compute.* Spot H200 at roughly $5/hr is a real bottleneck. Q1-Q3 together are approximately 160 H200-hours, feasible at solo-burn rate over four to six weeks. Q4 (the RLHF analogue) is not, at any plausible solo budget. (2) *Theory.* The author's training is empirical and engineering. Loss-landscape analysis, NTK regression, fixed-point characterization of the bootstrap operator, and singular learning theory framings are on the self-study list, not the active toolkit. The empirical setup is small enough to support those analyses; we do not yet have the machinery to extract a clean story from them. (3) *Time and focus.* Solo undergrad with two adjacent active projects (Parchment Labs, agentbreed). A collaborator with advisor backing converts this from a side empirical investigation into a primary line of work, which roughly triples the throughput.

Expansion plan. In rough order, if resources align: (i) Q1 first, in 2-4 weeks, because the tree-maze caveat is the most important threat to the existing claim. (ii) Q2 ablations in parallel with Q1, sharing infrastructure; 1-2 additional weeks once Q1 is queued. (iii) Q3 (offline-RL methods, especially CQL) next, ideally with input from a researcher in that area; 4-6 weeks. (iv) Q5 (theoretical framing) is gated on collaboration, not on time. (v) Q4 (RLHF analogue) is gated on a frontier-lab partner with API budget and, ideally, model access; we treat it as future work with a deliberately narrow empirical scope.

What we bring to a collaboration. Empirical execution capacity; working infrastructure (compute pipeline, evaluation harness, statistical machinery already built); reproducibility hygiene (pre-

registered protocols, public deviation logs, all configurations and seeds in version control); willingness to do the unglamorous work that produces clean negative results.

What we are looking for. A senior research collaborator with the theoretical machinery to interpret loss-landscape and bootstrap-stability dynamics; or institutional resources to run Q4 at meaningful scale; or experience taking a careful preliminary result through a top-venue submission. A smaller-scope mentor for any one of the open questions in §2 is also welcome.

Contact: tejas.naladala@gmail.com. Code, configurations, trajectories, and seeds: github.com/tejasnaladala/maze-rl-baselines.

References

Baird, L. (1995). Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of ICML 1995*.

Chevalier-Boisvert, M., Willems, L., and Pal, S. (2018). Minimalistic gridworld environment for OpenAI Gym (MiniGrid). github.com/Farama-Foundation/Minigrid.

Greenblatt, R., Roger, F., Krasheninnikov, D., and Krueger, D. (2024). Stress-testing capability elicitation with password-locked models. *arXiv:2405.19550*.

Hester, T., Vecerik, M., Pietquin, O., et al. (2018). Deep Q-learning from demonstrations. In *Proceedings of AAAI 2018*.

Hubinger, E., van Merwijk, C., Mikulik, V., Skalse, J., and Garrabrant, S. (2019). Risks from learned optimization in advanced machine learning systems. *arXiv:1906.01820*.

Kumar, A., Zhou, A., Tucker, G., and Levine, S. (2020). Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems 33*.

Kostrikov, I., Nair, A., and Levine, S. (2022). Offline reinforcement learning with implicit Q-learning. In *Proceedings of ICLR 2022*.

Langosco, L., Koch, J., Sharkey, L., Pfau, J., and Krueger, D. (2022). Goal misgeneralization in deep reinforcement learning. In *Proceedings of ICML 2022*.

Pomerleau, D. (1989). ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems 1*.

Shah, R., Varma, V., Kumar, R., et al. (2022). Goal misgeneralization: Why correct specifications aren't enough for correct goals. *arXiv:2210.01790*.

Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

Tsitsiklis, J. N., and Van Roy, B. (1997). An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42(5), 674-690.